Modul 2

Rendering Pipeline & Primitif 2D

Praktikum Grafika Komputer (Program Sarjana)

S1 Informatika – Fakultas Teknik – Universitas Muhammadiyah Surabaya

Topik Utama:

Light Transport & Rendering Pipeline – Forward (Object-Order) vs Backward (Image-Order) – Framebuffer & Konvensi Half-Pixel / Sample-Centre – Rasterisation Sebuah Garis (DDA, Bresenham) – Rasterisation Sebuah Segitiga (Edge Function, Koordinat Barycentric) – Aturan Fill Top-Left – Menggambar dengan Pustaka Grafika (Canvas, WebGL, Three.js) – Legenda 2D yang Aksesibel – Rasterisation vs Biaya Per-Pixel – Kesalahan Umum

Ringkasan Modul

Modul ini adalah tempat pixel berhenti menjadi sekadar data (Modul 1) dan mulai *digambar*. Anda akan mempelajari **rendering pipeline** — urutan tetap yang dijalankan setiap GPU dan setiap renderer software untuk mengubah geometri menjadi sebuah framebuffer — dan dua cara mengaturnya: **forward** (object-order: loop atas segitiga, splat ke pixel — rasterisation) dan **backward** (image-order: loop atas pixel, tanyakan apa yang menutupinya — ray casting). Anda kemudian akan membangun primitif intinya **dari nol, dalam JavaScript polos tanpa framework**: sebuah **garis** lurus lewat DDA dan lewat **Bresenham** integer, dan sebuah **segitiga** terisi lewat **edge function** dengan **aturan top-left** sehingga dua segitiga yang berbagi sebuah edge tidak meninggalkan *celah maupun pixel yang tergambar ganda*. Anda akan menetapkan **konvensi sample-centre** — sebuah pixel (x, y) di-sampling pada pusatnya $(x+\frac{1}{2}, y+\frac{1}{2})$ — karena salah setengah pixel saja membuat setiap sprite dan UI menjadi buram, bug yang menjadi dasar insiden nyata modul ini. Lalu Anda mereproduksi gambar yang sama dengan sebuah **pustaka grafika** sungguhan (HTML5 Canvas / WebGL / Three.js) dan membangun sebuah **legenda yang aksesibel** yang tetap terbaca di bawah sebuah colour-vision deficiency yang disimulasikan. Artefak yang dipakai sepanjang mata kuliah ini berlanjut: **denah lantai** dan **siluet** candi

Pusaka3D, digambar sekali oleh rasteriser Anda dan sekali oleh pustakanya, dibandingkan *pixel demi pixel* dengan sebuah difference image dan sebuah angka PSNR. Modul ini mengirimkan sebuah **halaman WebGL ter-hosting** yang dibuka penilai di sebuah browser. Ini sesuai dengan **Sub-CPMK-2**: *"Mampu menjelaskan pipeline rendering & menampilkan primitif 2D/3D dengan pustaka grafika. (Modul 1–2)"*

Manfaat ke Depan. Menjadi bekal setiap renderer yang akan Anda sentuh dan mata kuliah *Sains Data & Visualisasi* (pipeline-nya, primitif 2D, warna yang aksesibel, WebGL); ini adalah persiapan langsung untuk **UTS** (Proyek Rendering 3D) — rasteriser segitiga berbasis edge-function Modul 3 dengan z-buffer adalah persis milik modul ini ditambah sebuah depth test.

Contents

1 Peta Jalan dan Implementasi **3**
1.1 Pipeline Pusaka3D (dan di Mana Modul 2 Berada) . . . 3
1.2 Rendering Pipeline — Rangkaian Bernomor . . . 4
1.3 Di Mana Tiap Konsep Modul 2 Muncul di Industri . . . 5
1.4 Aplikasi Unggulan, Konsep demi Konsep . . . 6

2 Capaian Pembelajaran **7**

3 Motivasi **8**

4 Tugas Studi Pendahuluan (Pra-Modul) **8**

5 Light Transport dan Dua Arah Rendering **11**

6 Framebuffer dan Konvensi Sample-Centre **11**

7 Rasterisation Sebuah Garis: DDA dan Bresenham **12**
7.1 DDA (Digital Differential Analyzer) . . . 12
7.2 Bresenham — Hanya Integer . . . 13

8 Rasterisation Sebuah Segitiga: Edge Function **14**

9 Aturan Top-Left — Shared Edge yang Watertight **16**

10 Contoh Terpandu Lengkap: Melakukan Rasterisation Sebuah Siluet Kecil dengan Dua Cara **16**

11 Software dan Tooling **18**
11.1 Struktur Proyek yang Direkomendasikan . . . 18
11.2 Satu Perintah per Tugas . . . 18
11.3 Listing Inti: Segitiga Edge-Function dengan Aturan Top-Left . . . 19

12 Studi Kasus: Denah Lantai Pusaka3D, Digambar Dua Cara **20**
12.1 Konteks dan Target . . . 20
12.2 Walk-through . . . 20
12.3 Bukti: From-Scratch vs Pustaka . . . 21

13 Pitfalls **22**
13.1 Sampling pada Sudut Pixel, Bukan Pusatnya . . . 22
13.2 Tanpa Aturan Top-Left . . . 22
13.3 Edge Function Floating-Point dengan `== 0` . . . 22
13.4 Winding Tidak Konsisten . . . 22
13.5 Overdraw dari Bounding Box yang Longgar . . . 22
13.6 Drift Sumbu-Y dan Origin Antara JSON dan Layar . . . 22
13.7 Retina / devicePixelRatio pada Export Pustaka . . . 23

14 Fondasi (Pahami dan Terapkan) **23**
14.1 Edge Function dan Luas Bertanda 23
14.2 Interpolasi Barycentric 23
14.3 Interval Half-Open dan Sample Grid 23
14.4 Anti-Aliasing Garis (Pandangan ke Depan) 23

15 Kegiatan Lab (Modul 2) **24**
15.1 Lab 1: Telusuri Denah Anda, Baca Referensinya, Refleksikan 24
15.2 Lab 2: Bangun Rasteriser-nya, Diagnosis Celahnya, Perkuat 24
15.3 Lab 3: Gambar Denahnya dengan Sebuah Pustaka, Putuskan, Ship 25

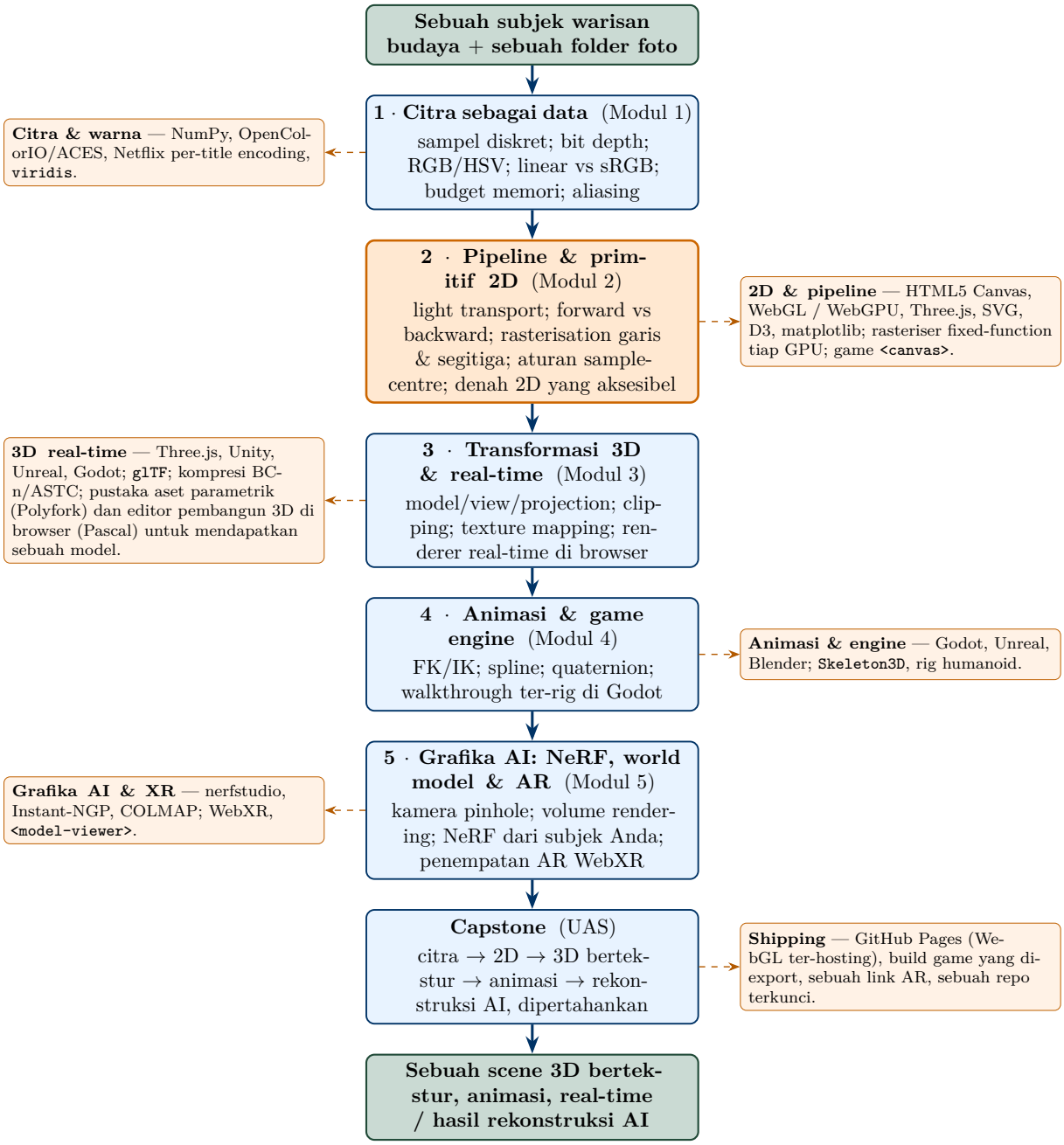
16 Tugas Analisis **27**

17 Catatan Penutup **28**

Peta Jalan dan Implementasi

Tujuan mata kuliah ini tidak berubah: mengambil **satu subjek warisan budaya pilihan Anda** dari sebuah folder **foto** menjadi sebuah **scene 3D bertekstur, animasi, real-time, atau hasil rekonstruksi AI**, mengimplementasikan primitif inti *dari matematikanya*. Modul 1 mengarakterisasi foto sebagai data. Modul 2 memegang langkah kedua: **pipeline dan primitif 2D** — bagaimana renderer mana pun mengubah bentuk menjadi pixel, dan dua primitif (garis, segitiga terisi) yang menjadi dasar semua yang lain dibangun.

Pipeline Pusaka3D (dan di Mana Modul 2 Berada)

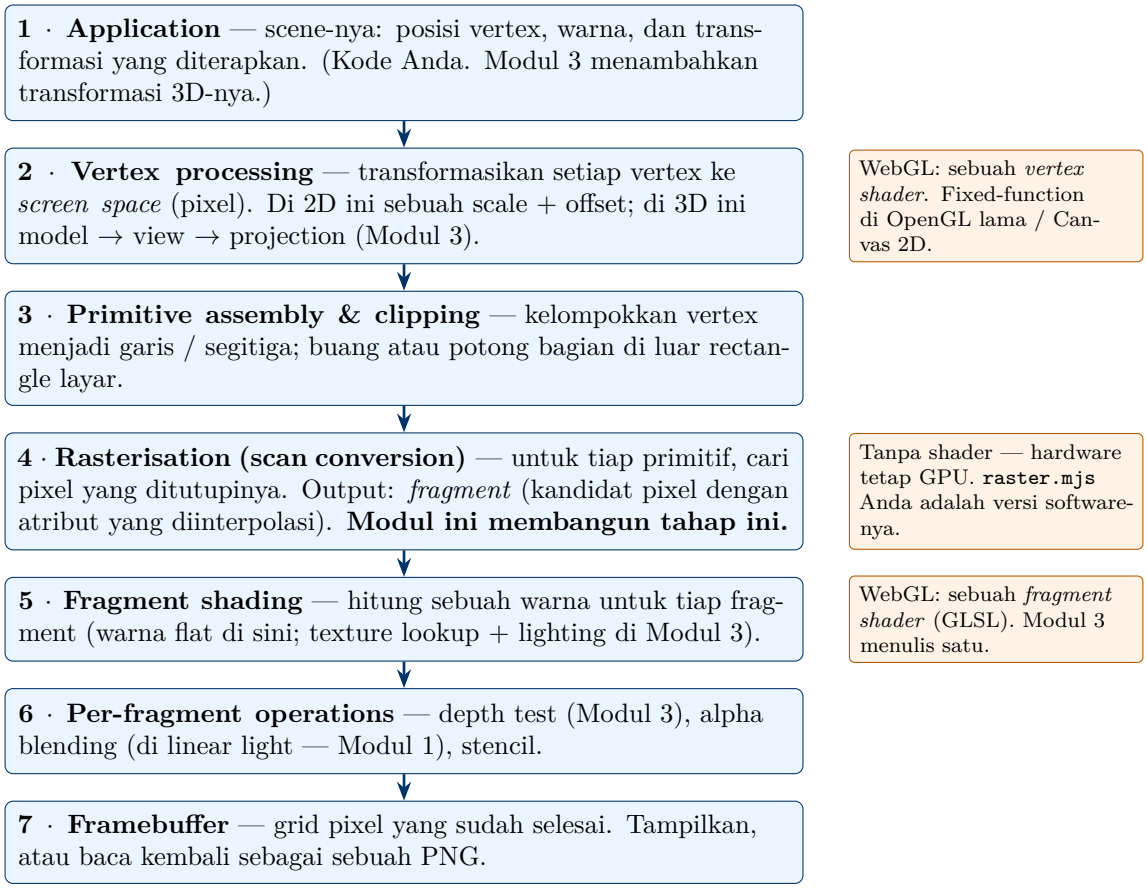


Biru solid: pipeline yang Anda bangun di sepanjang lima modul dan capstone. Oranye (langkah 2) adalah Modul 2.

Putus-putus oranye: alat nyata yang menjalankan langkah itu.

Rendering Pipeline — Rangkaian Bernomor

Setiap renderer, hardware atau software, menjalankan tahapan berurutan yang *sama* untuk mengubah sebuah scene menjadi sebuah framebuffer. Inilah diagram yang harus dihafal untuk UTS.



Tahap 2 dan 5 dapat diprogram (shader) pada sebuah GPU. Tahap 3, 4, dan 6 adalah hardware tetap yang Anda konfigurasi tapi tidak Anda kode. Modul 2 mengimplementasikan tahap 4 secara manual sehingga versi GPU-nya bukan kotak hitam.

Di Mana Tiap Konsep Modul 2 Muncul di Industri

Konsep Modul 2	**Penggunaan nyata & siapa yang bergantung padanya**	**Software / API tipikal**
Rendering pipeline	Model mental untuk setiap API grafika; pertanyaan wawancara klasik	OpenGL / WebGL, Direct3D, Vulkan, Metal, WebGPU
Forward vs backward (rasterise vs ray)	Memilih sebuah renderer; hybrid real-time ray tracing	GPU rasteriser vs OptiX / DXR / path tracer
Rasterisation garis (DDA, Bresenham)	CAD, editor vektor, plotter, osiloskop, rendering peta	`<canvas>`, renderer SVG, Mapbox / MapLibre
Rasterisation segitiga (edge function)	Satu-satunya primitif yang digambar semua hardware 3D; inti tiap GPU	Setiap GPU sejak ∼2001; `gl.drawArrays`, rasteriser software
Konvensi top-left / fill-rule	Mesh watertight; tanpa celah atau double-blend pada shared edge	Aturan rasterisation Direct3D & OpenGL; conservative raster
Konvensi sample-centre (half-pixel)	Sprite dan UI pixel-perfect; texture sampling yang benar	Aturan “texel-to-pixel” Direct3D 9; setiap engine 2D
Menggambar dengan sebuah pustaka	Dashboard, data-viz, UI editor, game	Three.js, D3, PixiJS, Skia, Cairo
Warna 2D yang aksesibel	Dashboard dan UI game aksesibel yang diwajibkan hukum	WCAG, `viridis`, ColorBrewer, mode colour-blind

Aplikasi Unggulan, Konsep demi Konsep

1. Algoritma garis Bresenham — 1962, dan masih ada di driver GPU Anda

Jack Bresenham mempublikasikan algoritma garis integer-nya di IBM pada 1962 untuk sebuah plotter digital. Ia memakai *hanya penjumlahan, pengurangan, dan pembandingan integer* — tanpa pembagian, tanpa floating point — sehingga ia berjalan pada hardware 1960-an dan masih menjadi dasar rasteriser font, konteks 2D `<canvas>`, CAD, dan mode garis tiap GPU. Anda mengimplementasikannya di §7.

2. Edge function — Pineda, 1988 — adalah cara setiap GPU mengisi sebuah segitiga

Paper Juan Pineda tahun 1988 "A Parallel Algorithm for Polygon Rasterization" menggantikan fill poligon scan-line dengan tiga **edge function**: ekspresi linear yang positif di satu sisi tiap edge. Sebuah pixel berada di dalam segitiga jika dan hanya jika ketiganya non-negatif. Ini trivial untuk diparalelkan (uji tiap pixel secara independen), dan itulah persis alasan GPU melakukannya dengan cara ini. `raster.mjs` Anda memakainya di §8, dan Modul 3 menambahkan sebuah nilai depth ke tiap hasil edge-function.

3. Offset half-pixel — sebuah bug yang terkirim selama bertahun-tahun

Sepanjang era Direct3D 9 (kira-kira 2002–2009) sebuah quad bertekstur yang sejajar layar harus digeser -0.5 pixel atau setiap blit keluar *blur secara bilinear* — Microsoft mempublikasikan seluruh halaman SDK, "Directly Mapping Texels to Pixels", tentang ini. Tak terhitung UI dan game 2D tampak sedikit lembek sampai developer mempelajari aturannya. Direct3D 10 mengubah konvensinya untuk menghilangkan jebakan ini. Pelajarannya — *tahu persis di mana sampel Anda mendarat* — ada di §6.

4. WebGL dan Three.js — pipeline-nya dalam sebuah tab browser

WebGL mengekspos pipeline tujuh-tahap yang sama ke JavaScript; Three.js membungkusnya sehingga Anda menulis `scene`, `camera`, `mesh` alih-alih buffer object dan GLSL. Jurnalisme data (The New York Times, Bloomberg), configurator produk, dan dashboard *Sains Data & Visualisasi* semuanya berjalan pada stack ini. Hasil Modul 2 Anda adalah sebuah halaman Three.js di GitHub Pages — hal yang sama, dalam skala mahasiswa.

Lanskap software untuk modul ini. From-scratch: **JavaScript polos**, dijalankan dengan **Node** untuk pengujian headless, tanpa framework, tanpa `<canvas>` (framebuffernya adalah sebuah typed array; fungsi yang sama menjalankan sebuah `<canvas>` browser lewat `putImageData`). Sisi pustaka: **Three.js** (WebGL) untuk viewer denah/siluet dan legendanya. Alat sehari-hari yang melakukan ini untuk Anda: konteks 2D `<canvas>`, SVG, PixiJS, Skia, Cairo, backend Agg milik matplotlib.

Capaian Pembelajaran

Pada akhir Modul 2, mahasiswa mampu:

LO1. Menggambar dan memberi label **rendering pipeline tujuh-tahap** dan menyatakan, untuk tiap tahap, apa yang masuk, apa yang keluar, dan apakah ia programmable atau fixed.

LO2. Menjelaskan rendering **forward** (object-order / rasterisation) vs **backward** (image-order / ray casting), dan menyebut sebuah beban kerja tempat masing-masing menang.

LO3. Menyatakan **konvensi sample-centre** — pixel (x, y) di-sampling pada $(x + \frac{1}{2}, y + \frac{1}{2})$ — dan menunjukkan apa yang dilakukan sebuah error $\frac{1}{2}$-pixel pada sebuah rectangle terisi dan sebuah blit.

LO4. Melakukan rasterisation sebuah garis dengan **DDA** dan dengan **Bresenham** integer, mendaftar setiap pixel secara manual untuk sepasang endpoint yang diberikan, dan menjelaskan mengapa Bresenham tidak butuh floating point.

LO5. Melakukan rasterisation sebuah segitiga terisi dengan **tiga edge function**, menghitung tanda edge-function pada sebuah titik sampel yang diberikan secara manual, dan mengaitkan nilai edge-function dengan **koordinat barycentric**.

LO6. Menerapkan **aturan top-left** sehingga dua segitiga yang berbagi sebuah edge menghasilkan **nol celah dan nol pixel tergambar ganda**, dan memverifikasinya dengan menghitung coverage.

LO7. Mengimplementasikan rasteriser garis dan segitiga **dari nol dalam JavaScript polos**, dan mencocokkannya **pixel-demi-pixel** dengan sebuah raster referensi.

LO8. **Mendiagnosis** sebuah fill segitiga yang retak di sepanjang shared edge *dari pixel yang dirender*, mengidentifikasi aturan top-left yang hilang, memperbaikinya, dan **membuktikan** watertightness-nya.

LO9. Menggambar **denah lantai dan siluet** Pusaka3D dengan sebuah pustaka grafika (Three.js), dengan sebuah **legenda yang aksesibel** yang bertahan pada sebuah deuteranopia yang disimulasikan.

LO10. **Memprediksi, lalu mengukur** biaya menggambar forward vs backward pada denah tersebut pada zoom 1× dan 10×, dan menetapkan satu pilihan dengan alasannya.

LO11. Mempublikasikan viewer-nya sebagai sebuah **halaman WebGL ter-hosting** yang dibuka penilai dari sebuah URL dan yang reproduksinya berasal dari repo ter-commit.

Motivasi

Sebuah renderer adalah sebuah fungsi dari *geometri* ke *pixel*. Ia tampak ajaib sampai Anda menulis dua baris yang benar- benar penting: "pixel mana yang disentuh segmen ini?" dan "pixel mana yang ditutupi segitiga ini?". Segala sesuatu yang lain — 3D, texture, lighting, animasi — adalah atribut yang menumpang di atas kedua jawaban itu.

Modul ini layak dipelajari dengan saksama karena:

- **Bisa diinterpretasikan.** Setelah Anda mengisi sebuah segitiga dengan tiga edge function integer, `gl.drawArrays` dan sebuah `Mesh` Three.js tidak lagi menjadi kotak hitam — Anda tahu apa yang dilakukan hardware antara data vertex Anda dan layar Anda.
- **Ini adalah baseline-nya.** Rasteriser from-scratch Anda adalah referensi yang dipakai untuk mengecek pustakanya. Saat sebuah garis Three.js di Modul 3 mendarat satu pixel meleset, Anda punya sesuatu yang eksak untuk mendiff-nya.
- **Ini adalah sebuah disiplin.** "Di mana persisnya sampel ini mendarat?" dan "apakah kedua segitiga ini berbagi sebuah edge dengan bersih?" adalah kebiasaan yang mencegah UI yang buram (insiden nyatanya) dan mesh yang retak (Modul 3).

Alur kerja yang Anda latih di sini — *tahu tahap pipeline-nya, tempatkan sampelnya secara presisi, bangun primitifnya, lalu cek pustakanya terhadap itu* — adalah persis alur kerja rasteriser z-buffer Modul 3 dan UTS.

Tugas Studi Pendahuluan (Pra-Modul)

Studi Pendahuluan (Minimum 4 Halaman)

Kumpulkan sebuah studi pendahuluan tertulis sebelum sesi praktikum (minimum 4 halaman, satu kolom, spasi baris 1,5). Gunakan kata, diagram, dan contoh kecil integer Anda sendiri. Kutip bab spesifik: Marschner & Shirley, *Fundamentals of Computer Graphics* (5e), Bab 3 (Raster Images) dan Bab 8–9 (The Graphics Pipeline, Rasterization); Hughes & van Dam *et al.*, *Computer Graphics: Principles and Practice* (3e), Bab 3 dan Bab 18 (Pipeline, Rasterization). Jawab minimal pertanyaan berikut.

Q1. Pipeline dan light transport.

Q1.(a) Gambar **pipeline tujuh-tahap** dari §1.2. Untuk tiap tahap tulis satu baris: input, output, dan programmable/fixed.

Q1.(b) Definisikan **light transport** dalam satu kalimat (cahaya meninggalkan sumbernya, terpencar pada permukaan, mencapai mata/kamera). Jelaskan bagaimana **rasterisation** dan **ray tracing** adalah dua arah menelusuri jalur yang *sama*.

Q1.(c) **Forward vs backward.** Rasterisation melakukan loop atas *primitif*; ray casting melakukan loop atas *pixel*. Untuk masing-masing, berikan satu beban kerja tempat ia jelas pilihan yang lebih baik dan jelaskan mengapa (petunjuk: jutaan segitiga kecil; vs satu bola reflektif).

Q2. Garis.

Q2.(a) **DDA secara manual.** Untuk segmen dari $(2, 1)$ ke $(9, 5)$: hitung Δx, Δy, jumlah step, dan increment x, y; lalu daftar *setiap* pixel (bulatkan y pada tiap step). Anda seharusnya mendapat 8 pixel.

Q2.(b) **Bresenham secara manual.** Segmen yang sama. Tunjukkan variabel keputusan $D = 2\Delta y - \Delta x$ dan update-nya pada tiap x; daftar pixel-nya. Pastikan cocok dengan (a).

Q2.(c) Mengapa Bresenham memakai **hanya integer** add/sub/compare, dan mengapa itu penting pada 1962 — dan mengapa itu masih penting pada sebuah GPU?

Q3. Segitiga.

Q3.(a) Definisikan **edge function** $E_{AB}(P) = (B_x - A_x)(P_y - A_y) - (B_y - A_y)(P_x - A_x)$. Apa yang dikatakan *tandanya*? Apa yang sama dengan *besarnya* (petunjuk: dua kali luas segitiga)?

Q3.(b) Untuk segitiga $A = (1, 1)$, $B = (6, 2)$, $C = (3, 7)$: hitung luas bertanda $2 \cdot \text{Area} = E_{AB}(C)$ dan nyatakan winding-nya (CW atau CCW). Lalu evaluasi ketiga edge function pada $P_1 = (3, 3)$, $P_2 = (5, 5)$, $P_3 = (1, 4)$ dan sebutkan titik mana yang berada di dalam.

Q3.(c) Definisikan **koordinat barycentric** $(\lambda_A, \lambda_B, \lambda_C)$ dari P dan tunjukkan bahwa mereka adalah ketiga nilai edge-function dibagi totalnya. Mengapa mereka bobot yang tepat untuk menginterpolasi sebuah warna atau sebuah depth di sepanjang segitiga?

Q3.(d) **Aturan top-left.** Dua segitiga berbagi edge BC. Jika Anda mengisi keduanya dengan "di dalam jika semua $E \geq 0$", pixel mana yang tergambar *dua kali*? Jika Anda memakai "> 0", pixel mana yang *terlewat*? Nyatakan aturan top-left yang memperbaiki keduanya.

Q4. Konvensi sample-centre dan sebuah insiden nyata. Baca halaman dokumentasi Microsoft Direct3D 9 **"Directly Mapping Texels to Pixels (Direct3D 9)"** (MSDN / Microsoft Learn) dan satu diskusi independen tentang "half-pixel offset" (misalnya catatan pixel-centre OpenGL vs Direct3D di OpenGL wiki, atau sebuah post-mortem game-dev). Lalu jawab:

Q4.(a) Di mana Direct3D 9 menempatkan pusat pixel-nya, dan di mana ia menempatkan koordinat *integer*-nya? Berapa offset yang dibutuhkan sebuah blit 2D akibatnya?

Q4.(b) Seperti apa persisnya tampilan citranya saat offset itu *hilang* — dan mengapa (kaitkan dengan sampling bilinear di antara empat texel)?

Q4.(c) Direct3D 10 mengubah aturannya. Apa konvensi modernnya (pusat pixel pada $x + \frac{1}{2}$)? Bagaimana itu menghilangkan jebakannya?

Q4.(d) Berikan tanggal perkiraannya dan sebutkan sumber Anda.

Q5. Menggambar dengan sebuah pustaka.

Q5.(a) Di Three.js, apa perbedaan antara sebuah `Line`, sebuah `Shape`/`ShapeGeometry`, dan sebuah `Mesh`? Mana yang akan Anda pakai untuk outline denah, siluet terisi, dan sebuah swatch legenda?

Q5.(b) Sebutkan tiga syarat konkret untuk sebuah legenda yang **aksesibel** (dari bagian CVD Modul 1) dan bagaimana Anda akan memenuhi masing-masing.

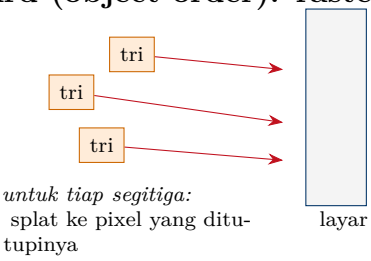
Kriteria penilaian: ketepatan pipeline; kebenaran aritmetika garis dan segitiga manual (Q2, Q3); kedalaman analisis insiden half-pixel (Q4); kejelasan diagramnya.

Light Transport dan Dua Arah Rendering

Light transport adalah fisika yang didekati sebuah renderer: cahaya meninggalkan sebuah sumber, memantul dari permukaan (sebagian diserap, sebagian terpencar), dan sebagiannya mendarat di kamera. Sebuah citra hasil render adalah sebuah estimasi, per pixel, berapa banyak cahaya yang datang di sepanjang ray melalui pixel itu.

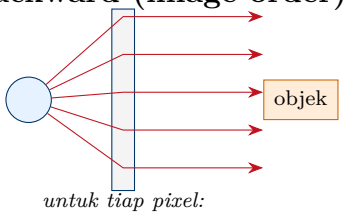
Ada dua arah untuk menghitungnya:

Forward (object-order): rasterisation



untuk tiap segitiga: splat ke pixel yang ditutupinya

Backward (image-order): ray casting



untuk tiap pixel: tembakkan sebuah ray, cari hit terdekat

	Forward / rasterisation	**Backward / ray casting**
Loop atas	primitif (segitiga)	pixel (ray)
Biaya berskala dengan	jumlah segitiga × luas pixel-nya	pixel × biaya intersection scene
Menang saat	banyak segitiga kecil, real-time, satu bounce (game, mata kuliah ini)	sedikit objek, permukaan lengkung, refleksi / refraksi / bayangan, kualitas offline
Hardware	setiap GPU, fixed-function	GPU ray core (RTX / DXR), path tracer CPU
Mata kuliah ini	Modul 2–4	perbandingan Modul 3; Modul 5 (NeRF adalah volume ray casting)

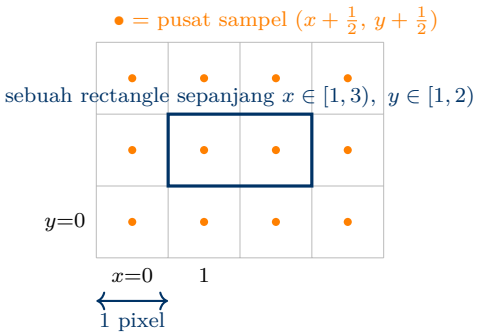
Mereka Menghitung Hal yang Sama

Sebuah rasteriser primary-visibility dan sebuah ray caster primary-ray menghasilkan citra yang *sama* (segitiga mana yang terdekat pada tiap pixel). Mereka berbeda dalam *urutan loop* dan karenanya berbeda dalam biaya dan efek tambahan apa yang murah. Engine modern bersifat **hybrid**: rasterise primary visibility, lalu tembakkan ray untuk bayangan dan refleksi.

Framebuffer dan Konvensi Sample-Centre

Framebuffer adalah grid outputnya — persis citra Modul 1: sebuah array byte `width` × `height` × `3`. Rasterisation *menulisnya*; sebuah display *membacanya*.

Satu aturan yang membuat semua orang tersandung: **pixel (x, y) adalah sebuah sampel titik yang diambil pada posisi kontinu** $(x+\frac{1}{2},\ y+\frac{1}{2})$**.** Pixel $(0, 0)$ menutupi kotak dari $(0, 0)$ sampai $(1, 1)$ dan di-sampling pada pusatnya $(0.5, 0.5)$.



Cara membacanya. Untuk mengisi rectangle $x \in [x_0, x_1)$, $y \in [y_0, y_1)$, Anda menyalakan pixel (x, y) jika dan hanya jika *pusatnya* $(x + \frac{1}{2}, y + \frac{1}{2})$ berada di dalam — yaitu $x_0 \le x + \frac{1}{2} < x_1$, yang untuk batas integer berarti $x = x_0, \ldots, x_1 - 1$. Memakai "$x_0 \le x \le x_1$" malah menyalakan satu kolom terlalu banyak di kanan. Sampling pada *sudut* pixel (x, y) alih-alih pusatnya menggeser seluruh bentuknya setengah pixel ke kiri-atas — yang, saat Anda menyalin sebuah texture, membuat tiap pixel output mendarat *di antara* empat texel sumber dan filter bilinear-nya mem-blur-nya. Itulah bug Direct3D 9.

Contoh E1 — Setengah Pixel, Diukur

Soal. Sebuah rectangle putih 2×2 pada sudut integer $(1, 1)$ di sebuah framebuffer hitam 6×6. (a) Pixel mana yang putih dengan aturan *pusat*? (b) Pixel mana jika Anda (secara salah) menguji *sudut* pixel $(x, y) \in [1, 3)$? (c) Jika sebagai gantinya Anda mem-blit sebuah texture 2×2 ke posisi *floating* $(1.0, 1.0)$ dengan sampling bilinear dan *tanpa* offset $-\frac{1}{2}$, berapa banyak texel sumber yang berkontribusi ke tiap pixel output?

Penyelesaian.

- (a) Pusat pada $(1.5, 1.5), (2.5, 1.5), (1.5, 2.5), (2.5, 2.5)$ berada dalam $[1, 3) \times [1, 3)$ $\Rightarrow$ pixel $\boxed{(1, 1), (2, 1), (1, 2), (2, 2)}$ — sebuah blok 2×2 yang tajam.
- (b) Sudut $(1, 1), (2, 1), (1, 2), (2, 2)$ *dan* $(3, \cdot), (\cdot, 3)$ teruji sebagai di dalam pada pengecekan half-open yang dilakukan di sudutnya, menyalakan sebuah blok 3×3 — satu baris dan satu kolom terlalu banyak, tergeser ke kiri-atas.
- (c) Tiap pusat output $(x+\frac{1}{2}, y+\frac{1}{2})$ memetakan ke sumber (x, y) tepat pada sebuah sudut texel, sehingga bilinear membaca $\boxed{4}$ texel dengan bobot $\frac{1}{4}$ masing-masing — sebuah blur seragam. Dengan offset $-\frac{1}{2}$-nya pusatnya memetakan ke sebuah pusat texel, bobot 1, dan salinannya eksak.

✓Seluruh modul ini bergantung pada mengetahui yang mana dari ini yang sedang Anda lakukan.

Rasterisation Sebuah Garis: DDA dan Bresenham

DDA (Digital Differential Analyzer)

Berjalan dari satu endpoint ke yang lain dalam step yang sama besar, menggerakkan sumbu *cepat* sejauh 1 dan sumbu *lambat* sejauh kemiringannya, dibulatkan ke pixel ter-

dekat.

$$\Delta x = x_1 - x_0, \quad \Delta y = y_1 - y_0, \quad \text{step} = \max(|\Delta x|, |\Delta y|),$$
$$x_k = x_0 + k \cdot \frac{\Delta x}{\text{step}}, \qquad y_k = y_0 + k \cdot \frac{\Delta y}{\text{step}}, \qquad k = 0, \ldots, \text{step}.$$

- $\Delta x, \Delta y$ — seberapa jauh garisnya berjalan pada tiap sumbu.
- step — berapa pixel panjang garisnya, di sepanjang sumbu dominannya.
- x_k, y_k — titik eksak (pecahan) pada step k; pixel-nya adalah $(\text{round}\, x_k,\ \text{round}\, y_k)$.

Cara membacanya. Ambil yang lebih panjang dari rentang horizontal dan vertikal; sebanyak itu +1 pixel akan digambar. Tiap step, tambahkan sebuah konstanta ke x dan sebuah konstanta ke y — satu di antaranya tepat ± 1 (sumbu cepatnya), yang lain adalah kemiringannya. Bulatkan keduanya untuk mendapat pixel-nya. DDA sederhana tetapi memakai sebuah pembagian dan penjumlahan floating-point per step.

Contoh E2 — DDA untuk $(2,1) \to (9,5)$ (garis yang dipakai sepanjang modul)

Soal. Daftar setiap pixel.
Penyelesaian. $\Delta x = 7$, $\Delta y = 4$, step $= 7$. Increment $x = 1$, increment $y = 4/7 \approx 0.5714$.

k	x_k	y_k (eksak)	round y_k	pixel
0	2	1.000	1	$(2,1)$
1	3	1.571	2	$(3,2)$
2	4	2.143	2	$(4,2)$
3	5	2.714	3	$(5,3)$
4	6	3.286	3	$(6,3)$
5	7	3.857	4	$(7,4)$
6	8	4.429	4	$(8,4)$
7	9	5.000	5	$(9,5)$

Pixel: $\boxed{(2,1), (3,2), (4,2), (5,3), (6,3), (7,4), (8,4), (9,5)}$ — 8 pixel, sebuah tangga run-2, run-2, run-2, run-1. ✓

Bresenham — Hanya Integer

Bresenham menyimpan sebuah **error term** integer D alih-alih sebuah y pecahan. Untuk sebuah garis dengan $0 \le \Delta y \le \Delta x$ (landai, menanjak):

```
let dx = x1 - x0, dy = y1 - y0;
let D = 2*dy - dx;           // decision variable, integer
let y = y0;
for (let x = x0; x <= x1; x++) {
  plot(x, y);
```

```
  if (D > 0) { y += 1; D += 2*(dy - dx); }   // step the slow axis
  else       {          D += 2*dy;         }   // stay
}
```

Listing 1: inner loop Bresenham, oktan landai menanjak ($0 \le \Delta y \le \Delta x$); raster.mjs menggeneralisasi ke semua delapan oktan

Cara membacanya. D melacak "seberapa jauh di atas atau di bawah pusat pixel garis sesungguhnya berada," diskalakan oleh $2\Delta x$ agar tetap integer. Mulai pada $D = 2\Delta y - \Delta x$. Pada tiap kolom: jika $D > 0$ garisnya sudah cukup jauh melenceng untuk menaikkan y, dan kita mengurangkan satu step penuh dengan menambahkan $2(\Delta y - \Delta x)$ (sebuah bilangan negatif); jika tidak garisnya tetap di baris ini dan kita menambahkan $2\Delta y$. Tanpa pembagian, tanpa float — tambah, bandingkan, tambah. Tujuh oktan lainnya sama dengan sumbu/tanda ditukar.

Contoh E3 — Bresenham untuk $(2,1) \to (9,5)$

Soal. Garis yang sama. Lacak D-nya.
Penyelesaian. $\Delta x = 7$, $\Delta y = 4$, $D_0 = 2(4) - 7 = 1$. $2\Delta y = 8$, $2(\Delta y - \Delta x) = -6$.

x	y	plot	lalu
2	1	$(2,1)$	$D = 1 > 0$: $y \to 2$, $D \to 1 - 6 = -5$
3	2	$(3,2)$	$D = -5 \le 0$: $D \to -5 + 8 = 3$
4	2	$(4,2)$	$D = 3 > 0$: $y \to 3$, $D \to 3 - 6 = -3$
5	3	$(5,3)$	$D = -3 \le 0$: $D \to -3 + 8 = 5$
6	3	$(6,3)$	$D = 5 > 0$: $y \to 4$, $D \to 5 - 6 = -1$
7	4	$(7,4)$	$D = -1 \le 0$: $D \to -1 + 8 = 7$
8	4	$(8,4)$	$D = 7 > 0$: $y \to 5$, $D \to 7 - 6 = 1$
9	5	$(9,5)$	selesai

Pixel $\boxed{(2,1), (3,2), (4,2), (5,3), (6,3), (7,4), (8,4), (9,5)}$ — **identik** dengan hasil DDA, hanya dengan operasi integer. ✓

Rasterisation Sebuah Segitiga: Edge Function

Untuk edge terarah $A \to B$ dan sebuah titik P, **edge function**-nya adalah

$$E_{AB}(P) = (B_x - A_x)(P_y - A_y) - (B_y - A_y)(P_x - A_x).$$

- Ia adalah komponen-z dari cross product $(B - A) \times (P - A)$: **positif** saat P berada di *kiri* $A \to B$, **nol** pada garisnya, **negatif** di kanan.
- Besarnya adalah **dua kali luas** segitiga ABP.
- Ia **linear** dalam P: Anda bisa mengevaluasinya sekali per segitiga lalu menambahkan sebuah konstanta per step pixel — itulah sebabnya ia bisa diparalelkan.

Tetapkan satu konvensi dan pegang: vertex didaftar sehingga **total** $E_{AB}(C) > 0$. Maka sebuah titik P berada di dalam jika dan hanya jika *ketiga* dari

$$E_{AB}(P) \geq 0, \qquad E_{BC}(P) \geq 0, \qquad E_{CA}(P) \geq 0.$$

Ketiga nilai itu, dibagi totalnya, adalah **koordinat barycentric** — bobot yang meng-interpolasi atribut per-vertex apa pun (warna sekarang; depth dan koordinat texture di Modul 3):

$$\lambda_A = \frac{E_{BC}(P)}{E_{AB}(C)}, \quad \lambda_B = \frac{E_{CA}(P)}{E_{AB}(C)}, \quad \lambda_C = \frac{E_{AB}(P)}{E_{AB}(C)}, \qquad \lambda_A + \lambda_B + \lambda_C = 1.$$

(Pada sebuah gambar "matematika" y-up, total positif berarti vertex-nya counter-clockwise; pada layar y-down, *rumus yang sama* dan *vertex yang sama* terbaca sebagai clockwise. Rumusnya tidak berubah — hanya katanya.)

Cara membacanya. Tiap edge function membelah bidangnya jadi dua di sepanjang satu edge. Berdirilah di sisi dalam ketiga half-plane dan Anda berada di dalam segitiganya. Seberapa jauh Anda dari sebuah edge (nilai edge-function-nya) relatif terhadap jarak vertex seberangnya (totalnya) adalah persis seberapa besar warna vertex seberang itu harus dihitung — itulah bobot barycentric-nya. Pada sebuah edge, satu bobot adalah 0; pada sebuah vertex, satu bobot adalah 1.

Contoh E4 — Isi Segitiga $A(1,1), B(6,2), C(3,7)$

Soal. (a) cek windingnya; (b) golongkan $P_1 = (3,3)$, $P_2 = (5,5)$, $P_3 = (1,4)$; (c) barycentric dari P_1.

Penyelesaian. Satu rumus sepanjang jalan: $E_{XY}(P) = (Y_x - X_x)(P_y - X_y) - (Y_y - X_y)(P_x - X_x)$.

(a) Total $= E_{AB}(C) = (6-1)(7-1) - (2-1)(3-1) = 30 - 2 = \boxed{28} > 0$, jadi urutan A, B, C adalah winding yang benar; luas $= 14$.

Ketiga edge function-nya, dijabarkan:

$$E_{AB}(P) = 5(P_y - 1) - 1(P_x - 1), \quad E_{BC}(P) = -3(P_y - 2) - 5(P_x - 6), \quad E_{CA}(P) = -2(P_y - 7) + 6(P$$

(b)

P	E_{AB}	E_{BC}	E_{CA}	vonis
$(3,3)$	8	12	8	semua ≥ 0: **di dalam**
$(5,5)$	16	-4	16	$E_{BC} < 0$: di luar
$(1,4)$	15	19	-6	$E_{CA} < 0$: di luar

(mis. pada $(3,3)$: $E_{AB} = 5(2) - 1(2) = 8$, $E_{BC} = -3(1) - 5(-3) = 12$, $E_{CA} = -2(-4) + 6(0) = 8$.)

(c) Untuk $P_1 = (3,3)$: $\lambda_A = E_{BC}/28 = 12/28 = 3/7$, $\lambda_B = E_{CA}/28 = 8/28 = 2/7$, $\lambda_C = E_{AB}/28 = 8/28 = 2/7$; jumlah $= 1$. Cek: $\frac{3}{7}(1,1) + \frac{2}{7}(6,2) + \frac{2}{7}(3,7) = \left(\frac{3+12+6}{7}, \frac{3+4+14}{7}\right) = (3,3) = P_1$. ✓

Bookkeeping Tanda — Pilih Satu Konvensi, Tuliskan

Tanda edge-function bergantung pada arah sumbu y Anda dan urutan vertex-nya. `raster.mjs` menetapkan satu dan menyatakannya di awal berkasnya: y **ke bawah** (layar); vertex diberikan sehingga total $E_{AB}(C) > 0$; "di dalam" = ketiga $E \geq 0$; aturan top-left memutuskan kasus "$E = 0$" (persis di edge). Jangan pernah mencampur dua konvensi dalam satu codebase.

Aturan Top-Left — Shared Edge yang Watertight

Saat dua segitiga berbagi sebuah edge (sebuah quad yang dipecah jadi dua, atau mesh apa pun), tiap pixel yang pusatnya berada *persis* pada edge itu punya edge function bernilai 0. Jika "di dalam" adalah "≥ 0", **kedua** segitiga mengklaimnya — ia tergambar dua kali (masalah untuk blending dan untuk stencil). Jika "di dalam" adalah "> 0", **tak satu pun** mengklaimnya — sebuah **celah** satu-pixel.

Aturan top-left: sebuah pixel yang pusatnya berada persis pada sebuah edge diisi hanya jika edge itu adalah sebuah edge **top** atau sebuah edge **left** dari segitiganya.

- Dengan konvensi y-down / total-> 0, edge $X \to Y$ dengan $\Delta = Y - X$ adalah: **top** jika $\Delta y = 0$ dan $\Delta x < 0$ (horizontal, interiornya di bawahnya); **left** jika $\Delta y > 0$ (ia berjalan ke bawah layar di sisi kiri segitiganya).
- Implementasi: sebuah *bias* per-edge senilai 0 jika edge-nya top/left, jika tidak -1; uji $E + \text{bias} \geq 0$. Sebuah pixel on-edge ($E = 0$) sekarang lolos untuk pemilik top/left-nya dan gagal untuk yang lain, sehingga ia diklaim **tepat sekali**.

Mengapa Anda Tidak Bisa Melewatkan Ini

Tanpa aturan ini siluet Pusaka3D Anda — dua segitiga per quad — akan punya sebuah seam diagonal samar berupa kecerahan ganda (dengan blending) atau pixel yang hilang (tanpa blending). Candi bertekstur Modul 3 akan menunjukkan celah warna latar di sepanjang tiap shared edge pada sudut kamera tertentu. Ini adalah hal pertama yang diperiksa lab diagnosis-dan-perbaiki.

Contoh Terpandu Lengkap: Melakukan Rasterisation Sebuah Siluet Kecil dengan Dua Cara

Kita melakukan rasterisation sebuah persegi 6×6 (sudut $(2, 2)$ dan $(8, 8)$, koordinat layar, y ke bawah) sebagai sebuah **outline** (4 segmen Bresenham) dan sebagai sebuah **fill** (dua segitiga terbagi di sepanjang diagonal utama), dan mengecek fill-nya watertight. Shared edge-nya adalah garis $y = x$, yang lewat *persis* melalui 6 pusat pixel $(2.5, 2.5), \ldots, (7.5, 7.5)$ — pixel yang harus diarbitrasi aturan top-left.

Segitiga 1 $= (2, 2), (8, 2), (8, 8)$; segitiga 2 $= (2, 2), (8, 8), (2, 8)$. Keduanya punya total > 0.

Langkah 1 — Outline dengan Bresenham

```
$ node draw_square.mjs --mode outline
4 segments (top, right, bottom, left) via lineBresenham()
pixels lit: 24      wrote out/square_outline.ppm
```

Empat segmen sejajar-sumbu masing-masing 7 pixel, berbagi 4 sudut: $4 \times 7 - 4 = 24$ pixel menyala. Hanya batasnya.

Langkah 2 — Fill dengan Dua Segitiga + Aturan Top-Left

```
$ node draw_square.mjs --mode fill
2 triangles, edge functions + top-left rule
coverage: 36 pixels, once = 36, twice = 0, gaps = 0
wrote out/square_fill.ppm
```

Ke-36 pixel interior (6×6) masing-masing terisi **tepat sekali**. Ke-6 pixel diagonal jatuh ke segitiga yang memiliki $y = x$ sebagai sebuah edge *left*.

Langkah 3 — Langgar Aturannya, Lihat Seam-nya

```
$ node draw_square.mjs --mode fill --no-top-left
coverage: 36 pixels, once = 30, twice = 6, gaps = 0     <-- 6
   double-drawn
$ node draw_square.mjs --mode fill --strict-greater
coverage: 30 pixels, once = 30, twice = 0, gaps = 6     <-- 6
   cracks
```

Keenam pixel pada $y = x$ adalah seluruh ceritanya: "≥ 0" tanpa bias menggambarnya di *kedua* segitiga (6 double — seam kecerahan-ganda di bawah blending); "> 0" menggambarnya di *tak satu pun* (6 celah berwarna latar). Hanya aturan top-left yang memberi 0 dan 0.

Langkah 4 — Diff Fill-nya Terhadap Referensi Ter-commit

```
$ node compare.mjs out/square_fill.ppm ref/square_fill_ref.
   ppm
same size 12x12   identical pixels: 144/144   PSNR: inf dB
   SSIM: 1.000   max |diff|: 0
```

Identik byte demi byte dengan referensinya — klaim yang lebih kuat daripada "tampak benar".

Poin-poin Kunci

- Outline = garis (Bresenham); fill = segitiga (edge function). Keduanya hidup di `raster.mjs`, tanpa framework.
- Aturan top-left adalah perbedaan antara sebuah fill watertight dan sebuah seam — di sini sebuah 0 vs 6 yang terukur.

- "Cocok dengan referensinya, PSNR = ∞" adalah standarnya, bukan "tampak baik-baik saja".

Software dan Tooling

Repo Modul 2 hidup di `code/Module2/`. Rasteriser from-scratch-nya adalah **JavaScript polos, dijalankan dengan Node** — tanpa framework, tanpa `<canvas>` (framebuffer-nya adalah sebuah `Uint8ClampedArray`; fungsi yang sama menjalankan sebuah `<canvas>` browser lewat `ctx.putImageData`). Sisi pustakanya adalah **Three.js** untuk viewer denah/siluet-nya.

Struktur Proyek yang Direkomendasikan

```
Module2/
├── package.json .. {"type":"module"}; tanpa dependency untuk bagian from-scratch
├── README.md ..... toolchain + baris "Verified"
├── raster.mjs ..... Framebuffer, lineDDA, lineBresenham, triangle (+top-left),
│                    triangleNaive, PPM r/w
├── plan.mjs ..... muat pusaka_plan.json; world->screen; renderPlan()
├── pusaka_plan.json ..... polyline denah lantai ter-commit (tetap)
├── draw_plan.mjs ..... CLI: render denah headless -> out/plan_scratch.ppm
├── draw_square.mjs ..... CLI: Contoh Terpandu Lengkap (outline / fill / coverage)
├── triangle_gap_bug.mjs ..... diagnosis-dan-perbaiki: celah/double shared-edge
├── bench.mjs ..... rasterise forward vs per-pixel backward, zoom 1x dan 10x
├── metrics.mjs ..... PSNR + SSIM global sederhana
├── tests.mjs ..... asersi E2/E3/E4 + watertightness + kecocokan referensi
├── ref/ ..... raster referensi ter-commit (PPM)
├── out/ ..... raster yang dibangkitkan
└── web/
    ├── index.html ..... hasil GitHub Pages-nya
    ├── viewer.mjs ..... Three.js: denah sebagai Line, siluet sebagai ShapeGeometry,
    │                    legenda aksesibel
    └── cvd.css ..... toggle filter SVG deuteranopia untuk legendanya
```

Satu Perintah per Tugas

Tujuan	Perintah
Jalankan setiap pengecekan from-scratch	`node tests.mjs` (atau `npm test`)
Render denahnya (from scratch)	`node draw_plan.mjs -w 800 -h 600`
Reproduksi contoh terpandu	`node draw_square.mjs -mode fill`
Tunjukkan bug shared-edge	`node triangle_gap_bug.mjs`
Timing forward vs backward	`node bench.mjs`
Serve halaman WebGL secara lokal	`npx serve web` (lalu buka URL-nya)

Listing Inti: Segitiga Edge-Function dengan Aturan Top-Left

```
// CONVENTION (do not mix another in): y is DOWN (screen). Pass vertices so
// the total edge(A,B,C) > 0. "inside" = all three edge functions >= 0, and
// the top-left bias decides the exactly-on-edge (== 0) case.
export const edge = (ax, ay, bx, by, px, py) =>
  (bx - ax) * (py - ay) - (by - ay) * (px - ax);

// top-left bias for edge X->Y: 0 if it is a top or left edge, else -1.
const topLeftBias = (xx, xy, yx, yy) => {
  const dx = yx - xx, dy = yy - xy;
  const isTop  = (dy === 0 && dx < 0);   // horizontal, interior below it
  const isLeft = (dy > 0);               // runs down the screen
  return (isTop || isLeft) ? 0 : -1;
};

export function triangle(fb, A, B, C, rgb, { topLeft = true } = {}) {
  const minX = Math.max(0, Math.floor(Math.min(A[0], B[0], C[0])));
  const maxX = Math.min(fb.w - 1, Math.ceil(Math.max(A[0], B[0], C[0])));
  const minY = Math.max(0, Math.floor(Math.min(A[1], B[1], C[1])));
  const maxY = Math.min(fb.h - 1, Math.ceil(Math.max(A[1], B[1], C[1])));
  const bBC = topLeft ? topLeftBias(B[0], B[1], C[0], C[1]) : 0;
  const bCA = topLeft ? topLeftBias(C[0], C[1], A[0], A[1]) : 0;
  const bAB = topLeft ? topLeftBias(A[0], A[1], B[0], B[1]) : 0;
  for (let y = minY; y <= maxY; y++) {
    for (let x = minX; x <= maxX; x++) {
      const px = x + 0.5, py = y + 0.5;              // SAMPLE CENTRE
      const wA = edge(B[0], B[1], C[0], C[1], px, py) + bBC;  // ~ lambda_A
      const wB = edge(C[0], C[1], A[0], A[1], px, py) + bCA;  // ~ lambda_B
      const wC = edge(A[0], A[1], B[0], B[1], px, py) + bAB;  // ~ lambda_C
      if (wA >= 0 && wB >= 0 && wC >= 0) fb.set(x, y, rgb);
    }
  }
}
```

Listing 2: raster.mjs (cuplikan) — triangle() dengan aturan top-left

Cara membacanya. Untuk setiap pixel di dalam bounding box segitiganya, ambil *pusatnya* $(x + 0.5,\ y + 0.5)$, evaluasi ketiga edge function di sana, tambahkan bias top-left tiap

edge, dan nyalakan pixel-nya jika ketiganya ≥ 0. Bias-nya adalah 0 untuk sebuah edge top atau left (sebuah pixel on-edge *dinyalakan* untuk segitiga itu) dan -1 untuk yang lain (*tidak*), sehingga sebuah pixel yang duduk persis pada sebuah shared edge diklaim oleh tepat satu dari kedua segitiganya. Memberikan `topLeft: false` mereproduksi bug celah/double untuk lab diagnosis-dan-perbaiki. Modul 3 menyimpan `wA, wB, wC`, menormalisasinya menjadi bobot barycentric, dan menginterpolasi depth serta koordinat texture dengannya.

Padanan Pustaka (untuk Referensi Saja)

`<canvas>` browser: `ctx.beginPath(); ctx.moveTo(ax,ay); ctx.lineTo(bx,by); ctx.lineTo(cx,cy); ctx.fill();` — konteks 2D-nya melakukan rasterisation untuk Anda (dan melakukan anti-alias pada edge-nya). Three.js: sebuah `THREE.Shape` → `ShapeGeometry` → `Mesh`. WebGL mentah: unggah 3 vertex, `gl.drawArrays(gl.TRIANGLES, 0, 3)` — rasteriser fixed-function GPU-nya menjalankan persis uji edge ini di hardware. Inti `raster.mjs` adalah untuk melihat ujinya.

Studi Kasus: Denah Lantai Pusaka3D, Digambar Dua Cara

Konteks dan Target

Tim Pusaka3D butuh sebuah gambar referensi 2D dari candinya: sebuah **denah lantai** (polyline outline) dan sebuah **siluet** (terisi), dengan sebuah legenda. Mereka menggambarnya **dua kali** — sekali dengan `raster.mjs` from-scratch, sekali dengan **Three.js** — dan membandingkan output pixel-nya, sehingga versi pustakanya dipercaya dan mahasiswa bisa melihat apa yang ditambahkan pustakanya (edge anti-aliased, posisi sub-pixel).

Input ter-commit-nya adalah `pusaka_plan.json`: sebuah daftar polyline berlabel yang tetap (dinding luar, cella dalam, tangga masuk, empat shrine sudut) dalam meter, plus sebuah scale dan origin. **Input yang dinilai adalah sebuah denah yang Anda telusuri dari bangunan Anda sendiri**; berkas ini adalah fallback-nya.

Walk-through

1. **Muat.** `plan.mjs` membaca `pusaka_plan.json`, mengonversi meter → pixel layar dengan sebuah scale/origin tetap, dan menghasilkan polyline.

2. **Gambar from scratch.** `node draw_plan.mjs -w 800 -h 600` melakukan rasterisation tiap polyline dengan `lineBresenham` dan mengisi siluetnya dengan `triangle`; menulis `out/plan_scratch.ppm`.

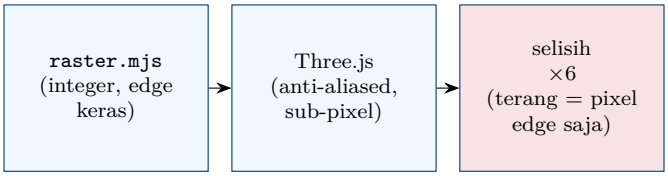
3. **Gambar dengan pustakanya.** `web/index.html` me-render `pusaka_plan.json` yang sama dengan `Line` dan `ShapeGeometry` Three.js pada ukuran yang sama; sebuah tombol meng-export canvas-nya menjadi `plan_three.png`.

4. **Bandingkan.** `node compare.mjs out/plan_scratch.ppm plan_three.png` melaporkan % pixel identik, PSNR, dan sebuah difference image. Harapkan PSNR yang tinggi tapi tidak tak hingga: pustakanya melakukan anti-alias pada edge yang dibuat keras oleh rasteriser integer Anda.

5. **Legenda.** Legenda viewer-nya memakai sebuah ramp **viridis** 5-langkah (luminansi bervariasi), sebuah label teks *dan* sebuah bentuk marker per kategori; sebuah toggle menerapkan sebuah filter SVG deuteranopia — kelimanya tetap berbeda (aturan Modul 1).

6. **Ship.** push `web/` ke GitHub Pages; penilai membuka URL-nya, melihat denah dan legendanya, meng-toggle filter CVD-nya, dan menjalankan ulang `node tests.mjs` dari repo-nya.

Bukti: From-Scratch vs Pustaka



Gambar (dihasilkan oleh `compare.mjs`, ditulis ke `out/`). Yang harus dicari: citra selisihnya gelap di mana-mana *kecuali* sebuah outline selebar satu pixel di sepanjang tiap edge — interiornya cocok persis; hanya anti-aliasing pustakanya yang berbeda. Beri caption angkanya: `PSNR` $\approx$ 30–40 `dB, interior identik`.

Hasil / Status yang Diharapkan

Setelah `npm test` dan perintah gambar pada toolchain yang tercatat:

- `node tests.mjs` mencetak `ALL CHECKS PASSED (10 ok)`: `lineDDA` dan `lineBresenham` pada $(2,1) \to (9,5)$ keduanya memberi daftar 8-pixel dari Contoh E2/E3; nilai edge-function pada $(3,3)$ adalah $(8,12,8)$ dan $(5,5),(1,4)$ berada di luar (Contoh E4); persegi 6×6 yang dipecah pada $y = x$ menutupi setiap pixel **tepat sekali** dengan aturan top-left (`once=36, twice=0, gaps=0`), `twice=6` tanpanya, `gaps=6` dengan "> 0"; fill `draw_square` identik byte demi byte dengan `ref/square_fill_ref.ppm`.
- `out/plan_scratch.ppm` ada pada ukuran yang diminta (800×600, 13 polyline, 37 segmen); `-fill` menambahkan siluetnya.
- `bench.mjs` mencetak waktu forward vs backward untuk zoom 1× dan 10×; pada host authoring forward ≈1 ms / 3 ms dan backward ≈520 ms / 380 ms (backward ∼100–500× lebih lambat — angkanya bervariasi antar mesin, arahnya adalah hasilnya).
- halaman WebGL-nya me-render denah dan sebuah legenda 5-kategori yang tetap berbeda di bawah toggle deuteranopia-nya.

Kegagalan Umum dalam Studi Kasus Ini

- **Off-by-one pada edge kanan/bawah** — melakukan loop `x <= maxX` tapi menguji sudut pixel bukan pusatnya. Pakai $(x + 0.5, y + 0.5)$.
- **Kebingungan sumbu Y** — JSON-nya meter y-up; layarnya pixel y-down. Balik sekali, di `plan.mjs`, dan dokumentasikan.

- **Winding tidak konsisten** — separuh segitiga Anda lenyap karena wound ke arah lain. Pilih CW-di-layar dan urutkan ulang vertex yang tidak sesuai.
- **Membandingkan ukuran berbeda** — canvas Three.js di-scale retina (2×). Paksa `renderer.setPixelRatio(1)` dan sebuah ukuran eksplisit sebelum meng-export.
- **PNG vs PPM** — `compare.mjs` membaca keduanya; jika sebuah decode hilang ia mengatakannya alih-alih melaporkan sebuah 0 palsu.

Pitfalls

Sampling pada Sudut Pixel, Bukan Pusatnya

Menggeser setiap bentuk setengah pixel ke kiri-atas; membuat blit menjadi blur bilinear. **Perbaikan:** evaluasi coverage-nya pada $(x + \frac{1}{2}, y + \frac{1}{2})$ selalu; untuk sebuah blit texture tambahkan offset sumber $-\frac{1}{2}$.

Tanpa Aturan Top-Left

Shared edge tergambar dua kali (seam double-blend) atau terlewat (celah). **Perbaikan:** bias top-left di §9; verifikasi dengan sebuah penghitungan coverage.

Edge Function Floating-Point dengan `== 0`

Pembulatan membuat "persis pada edge" tidak dapat diandalkan. **Perbaikan:** pakai koordinat integer jika bisa; jika tidak lipat keputusan top-left-nya ke dalam bias-nya sehingga Anda tidak pernah membandingkan dengan 0 mentah.

Winding Tidak Konsisten

Beberapa segitiga teruji sebagai selalu-di-luar. **Perbaikan:** satu konvensi winding, diny-atakan di awal berkasnya; balik urutan vertex saat dimuat jika perlu; atau uji `abs` untuk sebuah fill dua sisi.

Overdraw dari Bounding Box yang Longgar

Memakai seluruh framebuffer sebagai rentang loop untuk sebuah segitiga kecil membuang waktu. **Perbaikan:** clamp bounding box-nya ke segitiganya dan ke framebuffer-nya (seperti pada listing-nya).

Drift Sumbu-Y dan Origin Antara JSON dan Layar

Denahnya tampak terbalik atau di luar canvas. **Perbaikan:** satu fungsi konversi `worldToScreen()` dengan flip dan origin tertanam di dalamnya; uji dengan sebuah titik yang diketahui.

Retina / devicePixelRatio pada Export Pustaka

Canvas Three.js-nya 2× ukuran CSS-nya, sehingga diff-nya membandingkan grid yang tidak cocok. **Perbaikan:** `setPixelRatio(1)` dan sebuah `setSize(w,h,false)` eksplisit sebelum `toDataURL`.

Fondasi (Pahami dan Terapkan)

Edge Function dan Luas Bertanda

Mengapa Ini Penting untuk Modul 2

$E_{AB}(P)$ adalah dua kali luas bertanda $\triangle ABP$ dan komponen-z sebuah cross product. Ketiga nilai edge-function sebuah segitiga berjumlah (dengan tanda yang tepat) dua kali luasnya; dibagi totalnya mereka menjadi bobot barycentric. Modul 3 menginterpolasi depth dan UV dengan persis bobot ini.

Interpolasi Barycentric

Contoh E5 — Interpolasi Sebuah Nilai

Segitiga A, B, C membawa nilai $v_A = 10, v_B = 20, v_C = 40$. Pada P dengan barycentric $(\lambda_A, \lambda_B, \lambda_C) = (0.5,\ 0.25,\ 0.25)$ nilai terinterpolasinya adalah $0.5(10) + 0.25(20) + 0.25(40) = 5 + 5 + 10 = \boxed{20}$. Matematika yang sama menginterpolasi sebuah channel warna, sebuah z (Modul 3), atau sebuah koordinat texture (perspective-corrected di Modul 3).

Interval Half-Open dan Sample Grid

Mengapa Ini Penting untuk Modul 2

Isi $x \in [x_0, x_1)$ sebagai pixel $x_0 \le x + \frac{1}{2} < x_1$. Half-open di kedua sisi berarti dua rectangle yang bersebelahan tidak berbagi pixel apa pun — versi 1D dari aturan top-left. Setiap tiling bidang bergantung padanya.

Anti-Aliasing Garis (Pandangan ke Depan)

Mengapa Ini Penting untuk Modul 2

Bresenham memberi sebuah garis 1-pixel yang keras. Anti-aliasing (algoritma Wu, atau berbasis coverage) menetapkan intensitas tiap pixel menjadi fraksi yang ditutupi garisnya — dan blending itu harus dilakukan dalam **linear** light (Modul 1) atau garisnya tampak tipis dan gelap. MSAA Modul 3 adalah ide yang sama untuk edge segitiga.

Kegiatan Lab (Modul 2)

Lab 1: Telusuri Denah Anda, Baca Referensinya, Refleksikan

Lab 1 — Bawa Denah Anda Sendiri + Studi Pipeline + Refleksi Insiden

Tujuan: dapatkan input 2D Anda sendiri, tempatkan di dalam pipeline-nya, hubungkan dengan sebuah insiden nyata.

1. **Catat toolchain Anda**: OS, versi Node, browser + versi, versi Three.js, tanggal. Cantumkan di laporan dan `README.md`.
2. **Bawa input Anda sendiri.** Telusuri **denah lantai** bangunan warisan budaya pilihan Anda (dari sebuah foto, sebuah sketsa, atau sebuah pengukuran langsung) sebagai polyline. Commit `myplan.json` dalam skema yang sama dengan `pusaka_plan.json` (polyline berlabel, meter, scale, origin). Minimal: sebuah dinding luar, satu pembagian interior, sebuah pintu masuk. Jika Anda tidak bisa, pakai `pusaka_plan.json` dan katakan demikian.
3. **Tempatkan di dalam pipeline-nya.** Untuk denah *Anda*, tulis satu baris per tahap pipeline (§1.2) yang menjelaskan apa yang terjadi pada sebuah segmen dinding tunggal dari JSON sampai pixel menyala.
4. **Forward vs backward, di atas kertas.** Untuk sebuah zoom 10× dari denah Anda: prediksikan apakah melakukan rasterisation polyline-nya atau menguji tiap pixel layar terhadap tiap segmen lebih cepat, dan kira-kira berapa faktornya. Anda akan mengujinya di Lab 3.
5. **Baca, jangan dijalankan.** Baca salah satu dari: halaman "Vertex_Post-Processing"/rasterization OpenGL wiki, artikel rasterization WebGL fundamentals, atau source sebuah contoh `Line` Three.js. Setengah halaman: satu hal yang dilakukannya berbeda dari `raster.mjs` Anda, dan mengapa.
6. **Akurasi punya taruhan — sebuah refleksi kasus (≈setengah halaman).** Memakai halaman "Directly Mapping Texels to Pixels" Direct3D 9 dan satu sumber independen (Studi Pendahuluan Q4): (a) nyatakan offset-nya dan di mana pusat pixel-nya berada di D3D9 vs D3D10; (b) tunjuk baris persisnya di `raster.mjs` yang mengodekan konvensi pusatnya; (c) jelaskan citra uji (sebuah blit checkerboard, sebuah grid 1-pixel) yang membuat bug-nya jelas; (d) satu perubahan pada Studi Kasus modul ini yang akan menangkapnya. Sertakan tanggal dan sumber.

Lab 2: Bangun Rasteriser-nya, Diagnosis Celahnya, Perkuat

Lab 2 — Garis & Segitiga From-Scratch + Diagnosis-dan-Perbaiki

Tujuan: implementasikan primitifnya, cocokkan dengan sebuah referensi, perbaiki bug shared-edge-nya.

1. **From scratch (JS polos, tanpa framework).**
   - `lineDDA(A,B)` dan `lineBresenham(A,B)` — reproduksi daftar 8-pixel untuk $(2,1) \to (9,5)$ (Contoh E2/E3), dan tangani kedelapan oktannya (uji dengan 8 refleksi satu segmen).
   - `triangle(fb,A,B,C,rgb)` dengan edge function + aturan top-left — reproduksi tanda Contoh E4 pada $(3,3),(5,5),(1,4)$; lalu lakukan rasterisation segitiga yang dipakai sepanjang modul dan cocokkan `ref/tri_ref.ppm` pixel-demi-pixel.
   - `barycentric(A,B,C,P)` — kembalikan $(\lambda_A, \lambda_B, \lambda_C)$; cek jumlahnya 1 dan reproduksi Contoh E5.
2. **Diagnosis dan perbaiki (`triangle_gap_bug.mjs`).** Repo-nya menyertakan sebuah quad yang digambar sebagai dua segitiga dengan `triangleNaive` (tanpa aturan top-left). *Dari pixel yang dirender saja:* tunjukkan pixel mana yang tertutup dua kali (atau hilang), hitung jumlahnya, nyatakan penyebabnya, ganti ke `triangle`, dan **buktikan** 0 double dan 0 celah dengan sebuah citra coverage. Tulis diagnosisnya sebelum membaca perbaikannya.
3. **Cek sample-centre.** Isi sebuah rectangle 2×2 pada $(1,1)$ dengan aturan pusat dan sekali lagi menguji sudutnya; tunjukkan hasil 2×2 vs 3×3-nya (Contoh E1); jelaskan pergeseran kiri-atasnya.
4. **Terapkan ≥ 1 perbaikan yang didorong sebuah pitfall.** Salah satu dari: clamp bounding box segitiganya dan ukur percepatannya pada sebuah segitiga kecil di framebuffer besar; tambahkan anti-aliasing gaya-Wu ke `line` dan tunjukkan blend linear-light-nya vs blend naif; tambahkan sebuah auto-flip winding sehingga segitiga yang wound terbalik tetap terisi.
5. **Uji kesadaran-oktan.** Gambar sebuah asterisk 24-jari-jari dari sebuah titik pusat (garis pada tiap 15°) dan pastikan tiap jari-jari punya panjang yang sama dalam pixel ± 1 — menangkap sebuah oktan yang Anda salah.

Lab 3: Gambar Denahnya dengan Sebuah Pustaka, Putuskan, Ship

Lab 3 — Render Pustaka + Keputusan Forward/Backward + Halaman ter-Hosting

Tujuan: hasilnya adalah sebuah halaman WebGL ter-hosting dari denah *Anda* dengan sebuah legenda aksesibel, plus sebuah keputusan forward-vs-backward yang dipertahankan.

1. **Render pustaka.** Di `web/`, gambar `myplan.json` *Anda* dengan Three.js: outline polyline sebagai `Line`, siluet sebagai `ShapeGeometry`, pada ukuran tetap dengan `setPixelRatio(1)`. Export canvas-nya ke `plan_three.png`.
2. **Bandingkan dengan rasteriser Anda.** `node compare.mjs`

`out/plan_scratch.ppm` `plan_three.png`: laporkan % identik, PSNR, dan citra selisihnya. Jelaskan *setiap* pixel yang tidak cocok (semuanya seharusnya anti-aliasing edge).

3. **Keputusan dengan sebuah prediksi tertulis.** Sebelum menjalankan `bench.mjs`: prediksikan, untuk zoom 1× dan 10×, apakah forward (rasterise polyline-nya) atau backward (untuk tiap pixel layar, distance-test setiap segmen) lebih cepat, dan berapa faktornya. Lalu jalankan, tabulasikan ms untuk keduanya pada kedua zoom, **rekonsiliasikan**, dan tetapkan satu pilihan untuk sebuah viewer denah pan/zoom — dengan alasannya.

4. **Legenda aksesibel.** 5 kategori (dinding, cella, tangga, shrine, sumbu). Pakai sebuah ramp **viridis** (luminansi bervariasi), sebuah label teks, dan sebuah bentuk marker berbeda per kategori. Tambahkan sebuah toggle *Simulate deuteranopia* (filter SVG). Tunjukkan sebuah screenshot dengan filternya menyala dan mati; pastikan kelima kategori tetap terbedakan (aturan CVD Modul 1).

5. **Ship / serve / expose.** Publikasikan `web/` ke GitHub Pages. Hasilnya adalah **URL**-nya. Penilai membukanya, meng-toggle filternya, lalu meng-clone repo-nya dan menjalankan `npm test`. Bukan "clone contoh ini dan jalankan".

6. **Reproduksi-dari-bersih.** Clone baru → `npm install` (hanya `three` dan `serve`) → `npm test` hijau → `npx serve web` menampilkan halamannya. Perbaiki apa pun yang butuh campur tangan manusia; catat baris "Verified".

7. **Refleksi (setengah halaman).** Tahap pipeline mana yang diambil alih pustakanya dari tangan Anda, dan mana yang tetap harus Anda kerjakan sendiri dengan benar (winding, ukuran, colour space)? Apa yang pertama kali rusak jika denahnya punya 10,000 segmen alih-alih 50?

Optional Extension Labs (bacaan / eksplorasi saja — tidak dinilai)

- **Garis anti-aliased Wu.** Baca algoritma Xiaolin Wu 1991; tambahkan sebagai `lineWu` dan bandingkan kualitas edge-nya dengan Bresenham.
- **Sebuah rasteriser WebGL mungil.** Gambar segitiga yang dipakai sepanjang modul dengan WebGL mentah (3 vertex, sebuah vertex shader 6-baris, sebuah fragment shader 3-baris); pastikan GPU-nya menyalakan pixel yang sama dengan `triangle()`.
- **Fill scan-line.** Implementasikan fill poligon active-edge-table klasik dan bandingkan dengan pendekatan edge-function pada denahnya.
- **Conservative rasterisation.** Baca apa itu dan mengapa kode collision/voxel menginginkan "pixel mana pun yang disentuh segitiganya" alih-alih "pusat di dalam".
- **Teks/bentuk SDF.** Baca bagaimana signed-distance-field rendering menggambar glyph scalable yang tajam di GPU dengan satu texture.

Tugas Analisis

Tugas Analisis

Di sini Anda **menganalisis dan mempertahankan pekerjaan yang Anda lakukan di Kegiatan Lab di atas**. Pakai output Lab 1–3 *Anda*. Kumpulkan sebuah laporan terstruktur dan bersiaplah untuk menjalankan `node tests.mjs` dan menjelaskan satu commit Anda sendiri dalam sebuah viva lisan singkat.

A1. Dari Lab 1 — pipeline dan input.

- Sajikan `myplan.json` Anda dan walk-through per-tahapnya untuk satu segmen dinding (§1.2).
- Uji diagram pipeline-nya: untuk *satu* tahap, jelaskan apa yang akan terlihat rusak pada render denah Anda jika tahap itu dilewatkan.

A2. Dari Lab 2 — primitifnya, dicocokkan dengan sebuah referensi.

- **Reproduksi satu unit output secara manual:** pilih satu pixel yang dinyalakan `triangle()` Anda dekat sebuah edge; hitung ketiga edge function dan bias top-left-nya secara manual dan tunjukkan mereka cocok dengan kodenya (di dalam / on-edge / di luar).
- Tunjukkan `lineDDA` / `lineBresenham` memberikan daftar 8-pixel untuk $(2, 1) \rightarrow (9, 5)$, dan raster segitiganya cocok dengan `ref/tri_ref.ppm` (maks $|\text{diff}| = 0$).

A3. Dari Lab 2 — diagnosis dan perbaiki, dengan bukti.

- Bug shared-edge-nya: diagnosis pra-perbaikan Anda, hitungan pixel yang tergambar-ganda (atau hilang), penyebabnya (aturan top-left yang hilang), dan buktinya — sebuah citra coverage dengan `0 double, 0 celah` setelah perbaikannya. Jelaskan, memakai §9, edge mana yang mendapat bias 0 dan mana yang mendapat -1.

A4. Dari Lab 3 — pustaka vs from-scratch, dan keputusannya.

- Tabel `compare.mjs` (% identik, PSNR) dan citra selisihnya; jelaskan setiap pixel yang tidak cocok.
- **Prediksi** forward-vs-backward Anda, ms terukur pada zoom 1× dan 10×, rekonsiliasinya, dan pilihan ter-commit Anda dengan alasannya.
- **Pernyataan keterbatasan:** ≥ 2 kondisi konkret di mana pilihan Anda berbalik (mis. jumlah segmen denah; rentang zoom; menambahkan efek per-pixel; target GPU vs CPU), satu baris masing-masing.

A5. Dari Lab 1/3 — aksesibilitas.

- Legenda Anda sebelum/sesudah toggle deuteranopia; aturan yang Anda terapkan (ramp luminansi viridis + bentuk + label); pastikan kelima kategori tetap berbeda.

A6. Reproducibility & viva.

- Transkrip clone bersih: `npm install` → `npm test` hijau; URL GitHub Pages-nya.
- Bersiaplah untuk: menjalankan `node tests.mjs` secara langsung; membuka halaman ter-hosting-nya; menjelaskan satu commit dan menjawab "mengapa ini dan bukan itu?" dan "bagaimana jika target-nya WebGPU / sebuah canvas 4K?".

Struktur laporan: Input & penempatan pipeline → Primitif from-scratch & kecocokan referensi → Diagnosis-dan-perbaiki (diagnosis pra-perbaikan, hitungan, bukti) → Render pustaka & diff pixel → Keputusan forward/backward (prediksi, pengukuran, rekonsiliasi) → Aksesibilitas → Keterbatasan → Reproducibility (transkrip clone bersih + URL).
Penilaian mengikuti usulan *Rubrik Praktikum Modul 2 — Pipeline Rendering & Primitif 2D* (pembagian dari rubrik Modul 1 gabungan yang berlaku sekarang, menunggu persetujuan prodi; 5 baris, skor 1–4): *Pemahaman pipeline & light transport* **20%** · *Rasterisasi primitif 2D dari nol (garis/segitiga)* **30%** · *Render 2D & GUI dengan pustaka (WebGL/Three.js) + aksesibilitas warna* **25%** · *Kualitas kode & interpretasi* **15%** · *Ketepatan waktu* **10%**. Tiap kriteria diberi skor 1–4 dan Nilai = $\sum$(bobot % × skor/4) pada skala 0–100. Pengumpulan yang tidak bisa dibedakan dari sebuah repo starter yang belum diubah, atau dari sebuah penyelesaian LLM langsung atas soal ini, mendapat nilai nol pada baris "rasterisasi primitif 2D dari nol" dan "interpretasi" terlepas dari apakah kodenya berjalan.

Catatan Penutup

Perangkat Modul 2 adalah landasan langsung untuk UTS dan Modul 3:

- **Modul 3** menambahkan sebuah z ke setiap fragment yang dihasilkan `triangle()` Anda dan sebuah depth test di tahap pipeline 6 — raster edge-function yang sama, sekarang benar hidden-surface. Bobot barycentric yang Anda hitung di sini menginterpolasi depth dan koordinat texture (perspective-corrected) di sana.
- **Modul 3** juga menggantikan scale-dan-offset 2D pada tahap pipeline 2 dengan tumpukan matriks model → view → projection — tetapi tahap 3–7 tidak berubah.
- **Modul 4** meng-animasi-kan vertex yang masuk ke tahap 2; rasteriser di bawahnya tidak peduli.
- **UTS** (Proyek Rendering 3D) persis "pipeline Modul 2 + transformasi dan texture Modul 3" pada subjek Anda.
- **Sains Data & Visualisasi** memakai ulang separuh pustakanya: Three.js, sebuah legenda aksesibel, sebuah halaman ter-hosting.

Sebelum Anda menggambar apa pun, tanyakan pertanyaan Modul 2:

"Di tahap pipeline mana saya berada, di mana persisnya sampel ini mendarat, dan jika dua primitif bertemu di sini apakah pixel ini milik salah satunya — tepat satu?"